TECHNICAL PORTFOLIO

Physarum

Pattern Formation Simulation

Local sensing, persistent state and emergent field patterns
built through visual programming in Blender Geometry Nodes and shaders.



Cinematic presentation of the procedural field within an authored 3D environment.

Agents

LOCAL STATE

Field

SHARED MEMORY

Surface

SPATIAL OUTPUT

A technical study of how local rules produce a larger spatial pattern

01 / SYSTEM ARCHITECTURE

Agents, a field and a feedback loop

I built this Physarum-inspired visual programming project in Blender Geometry Nodes: point agents read and modify a shared scent field. Each agent carries a position and heading, samples three nearby locations, then updates its direction and moves. The grid retains the agents' deposits between evaluations. This creates changing spatial patterns from local rules rather than a pre-drawn trail. A separate shader turns the stored field into a visible surface. The project explores agent-based computation; the current implementation does not extract a shortest path.

1.1 Two representations of the same evolving system

Agent state	Point cloud	Position and normalized XY heading
Environmental state	Mesh vertices	Float attribute scent on the POINT domain
Rendered output	Material-assigned grid	Emission and displacement read scent

1.2 The actual organized node tree

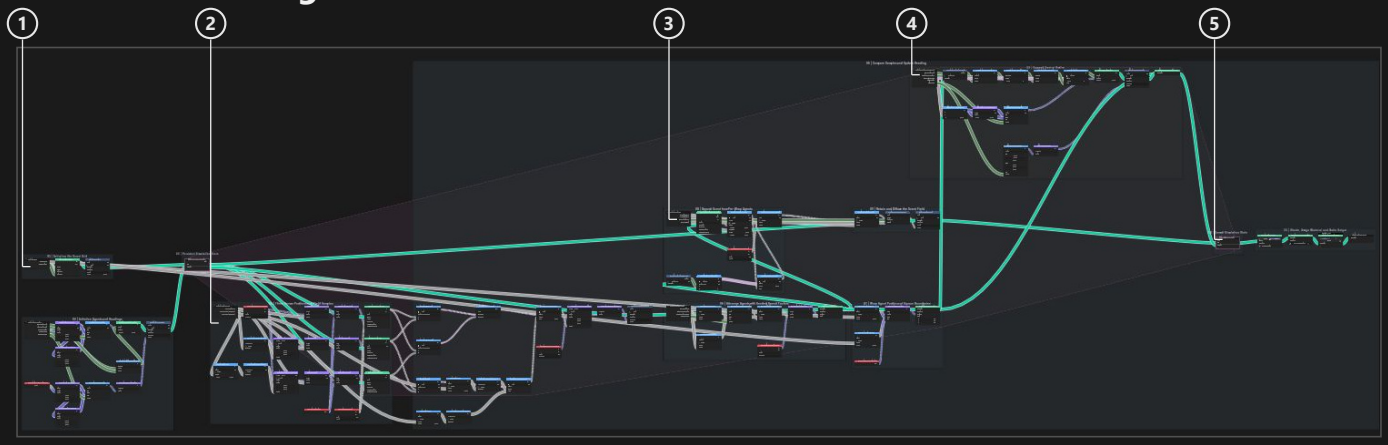


Figure 1. Overall visual program in Geometry Nodes. (1) initialize state, (2) sense and move agents, (3) update scent, (4) emit a capped batch, (5) commit state and output the field surface. The close views on following pages resolve the individual operations.

1.3 Ordering determines causality

Both branches read the previous simulation state. Existing agents move and deposit before new agents are joined at Simulation Output. New arrivals begin sensing and depositing on the next evaluation. With initial count zero, evaluation one ends with 150 agents but still has a zero scent field. The state is carried explicitly through the simulation zone; it is not reconstructed from the rendered image.

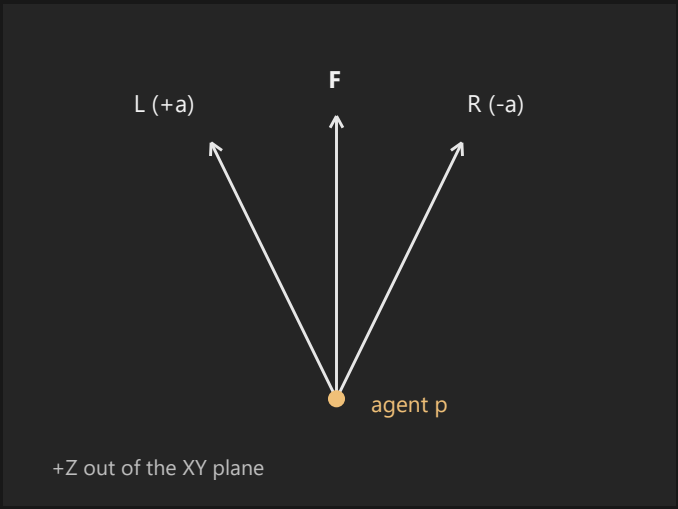
Inspected portfolio configuration: a 5-unit XY square, 1024 vertices per axis and a maximum of 100,000 agents. Resolution, initial population and emission rate are procedural controls.

02 / AGENT CONTROLLER

From spatial samples to a heading

A heading is a unit vector in the XY plane. The agent samples one location ahead and two locations rotated around Z. This separates perception from movement: sensor distance controls how far the agent reads, while turn angle controls its response. The samples interpolate scent from the nearest mesh surface [2], rather than selecting a single nearest grid vertex.

2.1 Constructing the three sensor positions

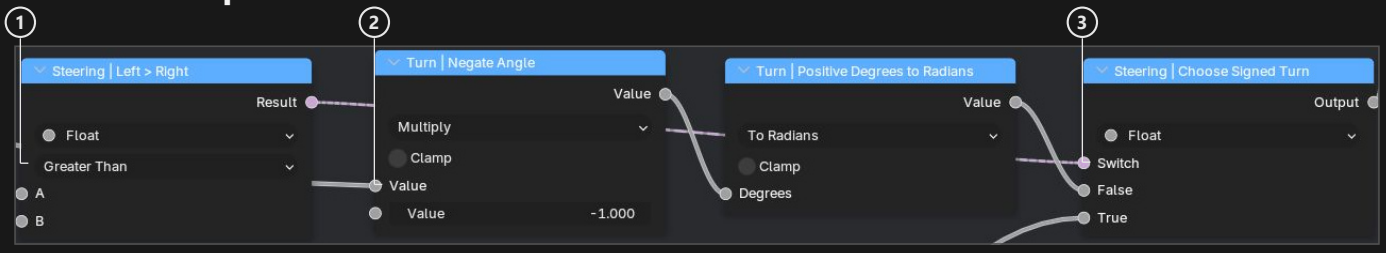


s_F = p + d h
s_L = p + d Rz(+a) h
s_R = p + d Rz(-a) h

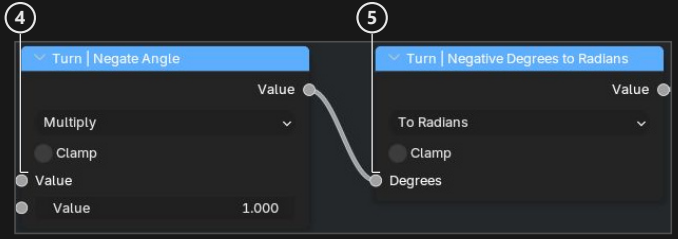
d = 0.2 units
a = 25 degrees
Rz is rotation about Z.

Figure 2. Sensor construction in local XY coordinates. F, L and R denote the sampled scalar values at these positions.

2.2 The comparison rule in this tree



Turn direction follows the +1 and -1 multipliers, rather than the node labels.



F >= L and F >= R : turn = 0

otherwise: +b if L > R, else -b

Figure 3. (1) compare side samples, (2) form -b for False, (3) select the signed turn; (4) form +b for True and (5) convert to radians. The forward-largest override follows this branch.

At b = 15 degrees, 14 isolated checks confirmed attraction and forward preference. Side ties with lower forward scent select -b; the rotated heading is normalized before storage.

03 / POPULATION AND MOTION

A bounded emitter, a persistent population

The initial point count is zero. A central emitter adds up to 150 agents per evaluation until the configured capacity is filled. The cap limits only new emission: existing points remain in the loop and continue updating. Limiting the final batch, rather than testing a Boolean threshold alone, prevents overshoot when capacity is not a multiple of the batch size.

3.1 Capacity is calculated before point creation

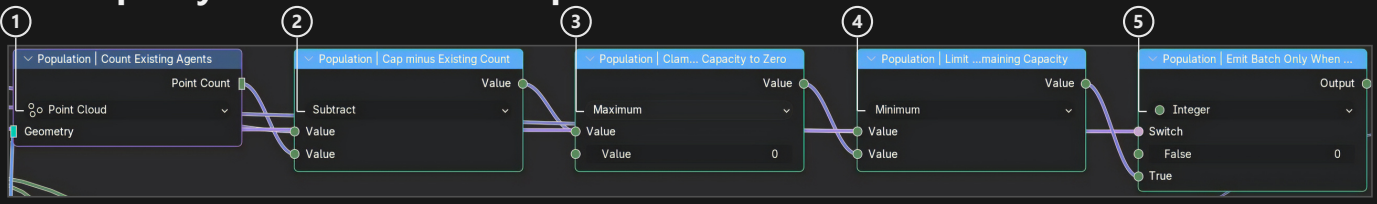


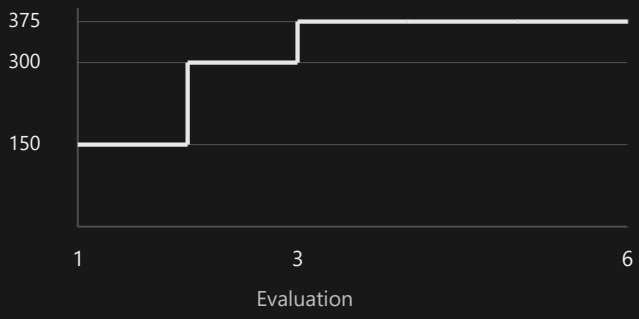
Figure 4. (1) count existing points, (2) subtract from cap, (3) clamp remaining capacity to zero, (4) take the smaller of rate and capacity, (5) gate emission. The result drives Points Count.

```
remaining = max(C - P, 0)

batch = min(B, remaining) if emission_enabled else 0
```

P = existing population; C = cap; B = emission rate.

3.2 Reaching capacity does not freeze the state



Measured cap test: C = 375, B = 150.
The count rises to 375 and stays there while agents keep moving and depositing. Lowering C does not delete existing points. Movement per evaluation is 0.02 units times a fixed random speed factor in [0.6,1.4]. Delta Time is unused.

3.3 Initialize every emitted point before joining

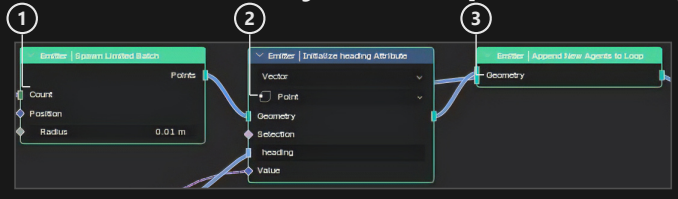


Figure 5. (1) Points, (2) initialize heading, (3) join with existing agents. The population controls sit in the first modifier panel. Initial population is also clamped to C.

The batch receives XY jitter within [-0.2,0.2] per axis and a normalized random heading. Store Named Attribute creates **heading** on points before Join Geometry appends them to state. The seed is fixed across evaluations; spawn patterns can repeat.

04 / FIELD UPDATE

Deposit, retain and mix spatial memory

The field records where agents have passed. Its update combines a distance-based deposit with the previous scent, then applies retention and neighbor mixing. The geometry branches matter: deposition uses moved positions before boundary wrapping, while the wrapped agents are what the next state retains.

4.1 A nearest-distance footprint

$$D(x) = \max(0, 1 - \text{distance}(x, \text{nearest_agent}) / r)$$

For a nonempty agent set and positive radius r. The empty-set guard returns D(x) = 0.

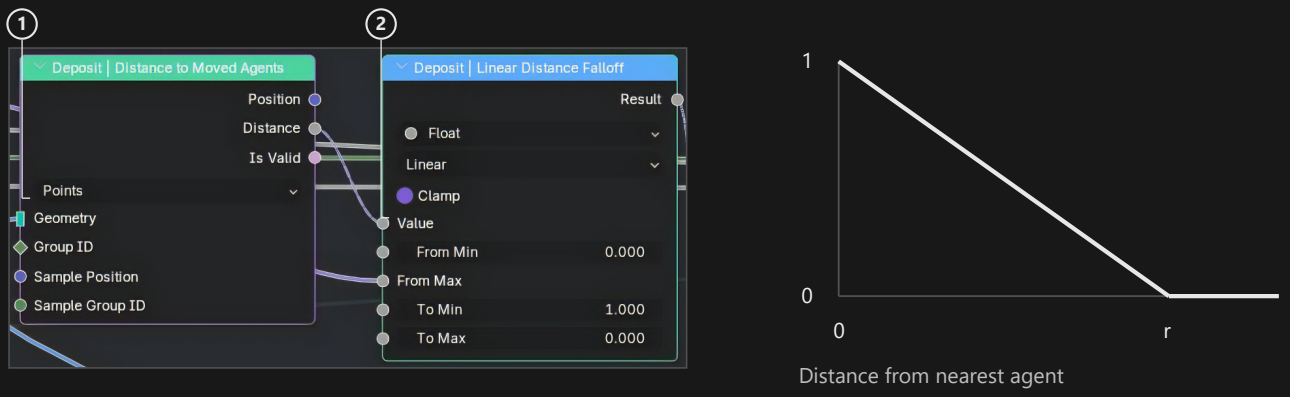


Figure 6. (1) Geometry Proximity reads the pre-wrap point set. (2) clamped Map Range gives a linear falloff. Multiple overlapping points do not add separate deposits: the closest distance determines D.

4.2 Accumulation and neighborhood operations



Figure 7. (1) multiply previous scent plus deposit by retention, (2) Blur Attribute mixes connected neighbors [3], (3) store float scent on grid vertices.

$$S_{next} = \text{Blur}(\text{retention} \times (S + D), \text{iterations}, \text{weight})$$

Retention 0.95 also scales the new deposit. Two blur iterations use Weight 0.1; this is a mesh-neighbor operation, not a calibrated diffusion coefficient. At the 5-unit, 1024-vertex setting, spacing is 5/1023 = 0.00489 units, close to the 0.005 deposit radius. Resolution and radius therefore control the footprint together.

05 / GEOMETRY TO SHADER

Turning a scalar field into a surface

The generated grid carries the simulation result as the named attribute scent. The material reads it for emission and object-space displacement. This makes the field visible through brightness and relief while preserving its numerical state in Geometry Nodes. The lab scene, lights and camera provide presentation rather than simulation feedback.

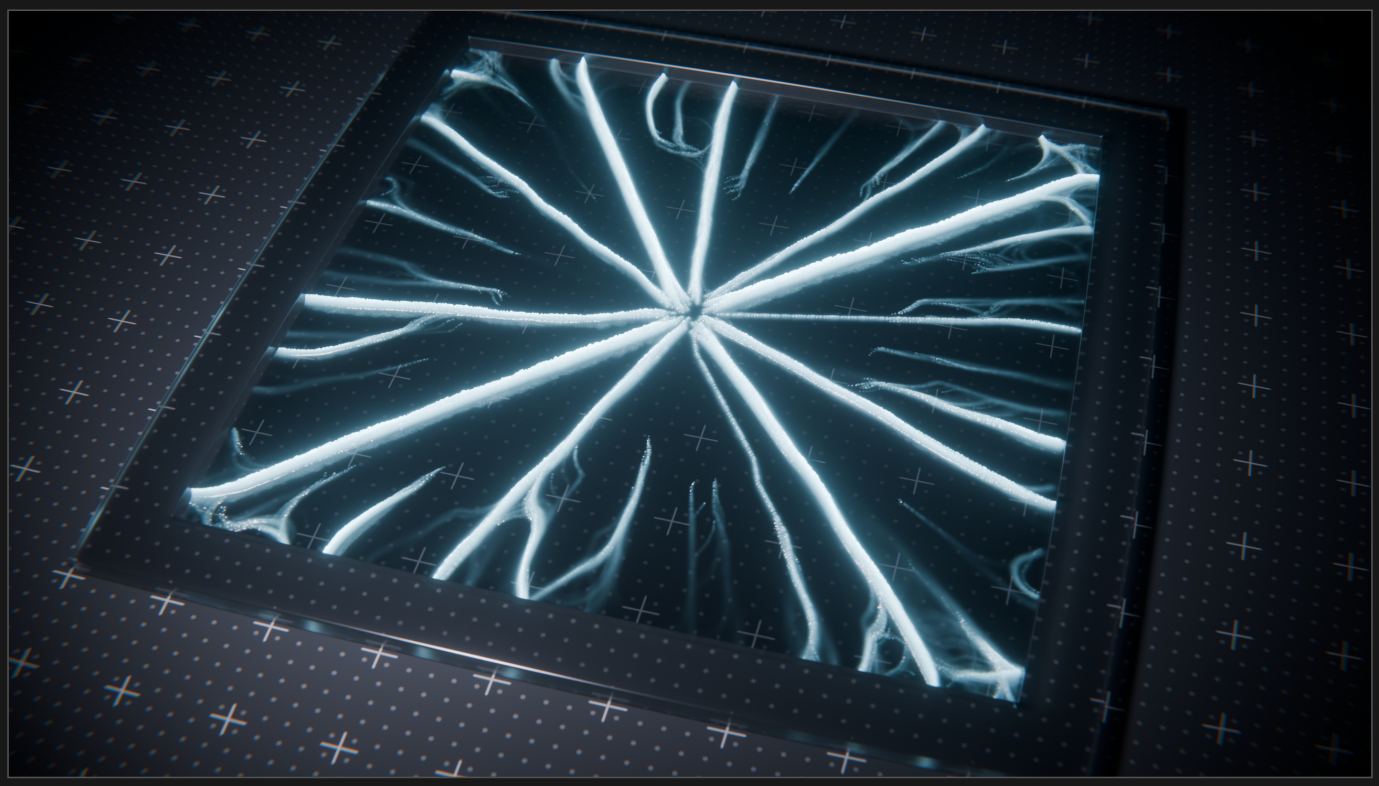


Figure 8. Render of the simulation, showing the field's trails and relief. Lighting and material response also affect the final appearance.

5.1 The attribute is the contract between both trees

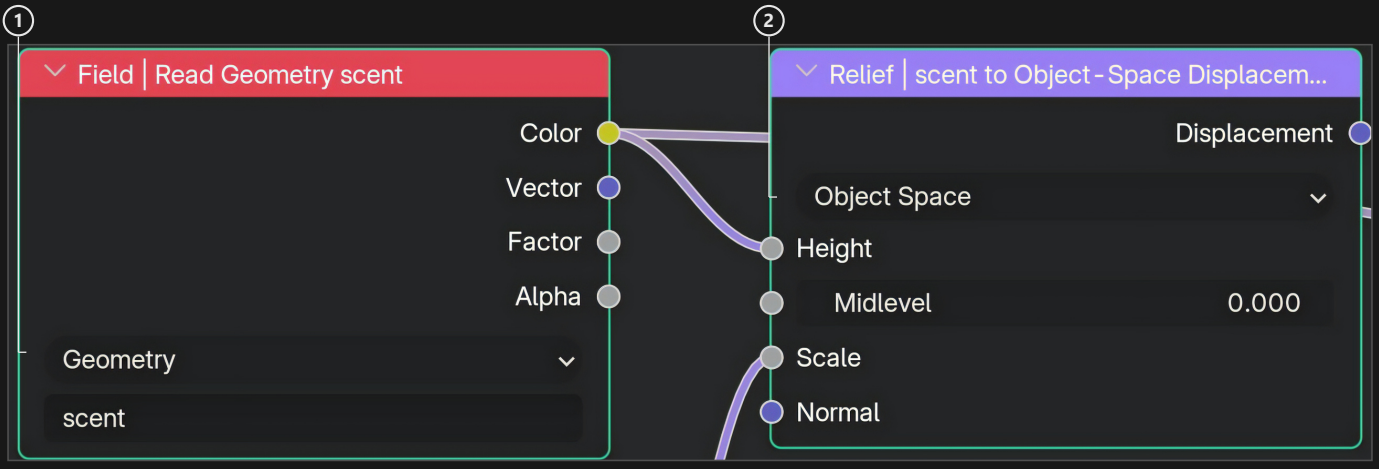


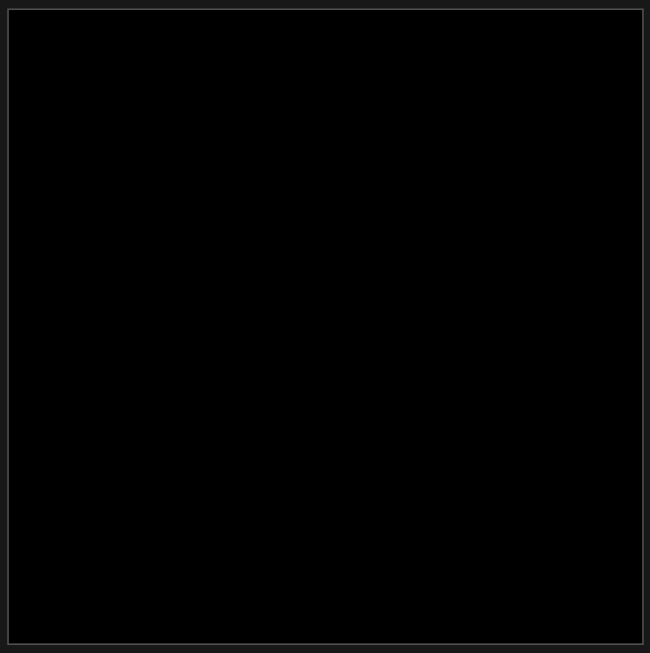
Figure 9. (1) Attribute reads geometry scent. Its Color output feeds both emission strength and (2) displacement Height through the existing socket conversions. Midlevel is zero; the inspected displacement scale is 0.01.

06 / OBSERVED DEVELOPMENT

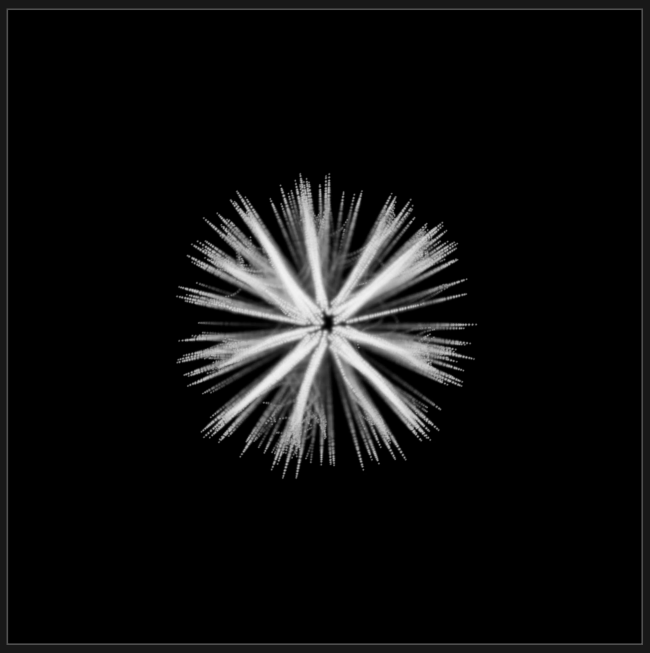
A field that develops over time

The grayscale clip shows the field expanding from the central region, with later frames developing broader, curved trails across the square. These frames make the temporal progression visible in a static document. They show field appearance rather than individual point trajectories.

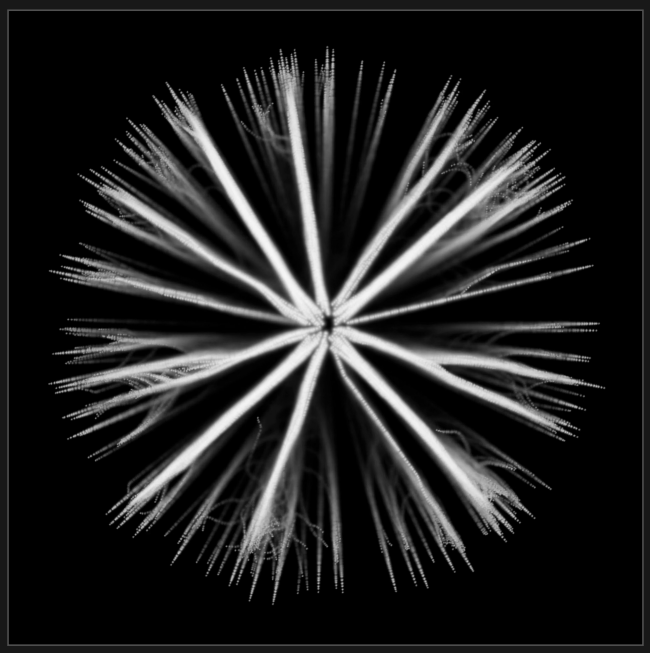
FRAME 001 0.00 s



FRAME 050 2.04 s



FRAME 100 4.12 s



FRAME 250 10.38 s

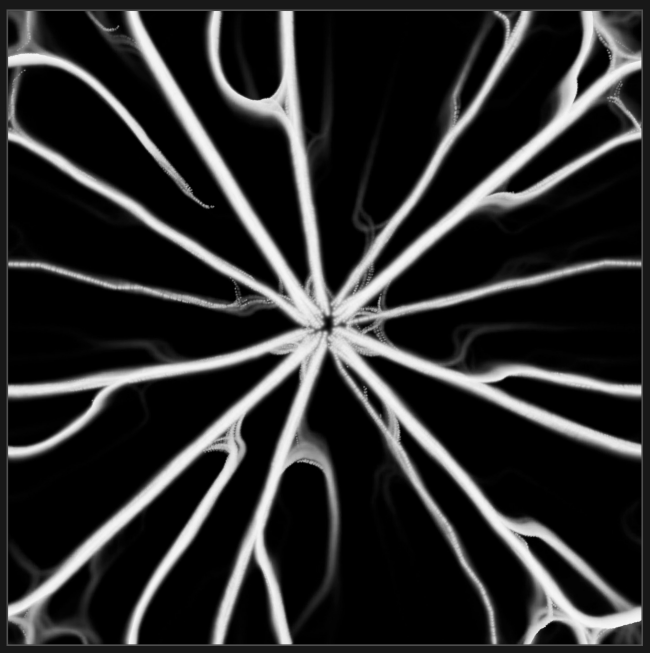


Figure 10. Frames 1, 50, 100 and 250 from the 250-frame, 24 fps clip (10.42 seconds). The first frame is blank; later frames show developing trails. Times indicate clip positions, not computation time.

07 / SCOPE, EVIDENCE AND NEXT STEPS

What this system does and does not do

This is a procedural agent-and-field model. It produces an evolving scent field; it does not extract a route or establish path optimality. I distinguish the visible pattern from what the algorithm has actually computed.

7.1 Capabilities and boundaries

Local state and feedback	Agents sample a shared field and update stored position and heading.
Procedural structure	Population, sensing, movement, field resolution and rendering are adjustable.
No extracted shortest path	No start/goal objective, obstacle map or route-cost comparison is present.
No learned or biological model	No training, neural network, physical fluid model or biological validation.

Only agents wrap at the boundary; sensing and grid blur are not periodic. The side response turns toward stronger scent. A separate noise-to-bump branch adds visual microtexture and does not contribute to scent.

7.2 Checks on the implemented mechanics

Fourteen isolated steering checks passed on a 32-by-32 grid, confirming left/right attraction, forward preference and defined ties. Nine population tests used a fresh 64-by-64 grid; the 375-agent probe reached 150, 300, 375, then held its count while agents kept moving. These are sampled checks, not long-run or full-resolution benchmarks. The portfolio grid remains 1024-by-1024.

7.3 Research connection and what I would build next

Physarum-inspired agent models connect local sensing to emergent transport patterns [1]. The relevant connection to robotics is the computational structure: repeated observations, persistent local state and interaction through a shared environment. My next step would be to quantify parameter sensitivity, add explicit goals and obstacles, then extract a graph and compare route cost with a conventional planner. That would turn a visual field experiment into a measurable navigation problem.

REFERENCES AND IMPLEMENTATION BASIS

[1] Jones (2010). Characteristics of pattern formation and evolution in approximations of Physarum transport networks.

[2] Blender Manual. Sample Nearest Surface: interpolated values on a source mesh.

[3] Blender Manual. Blur Attribute: mixing values over connected neighbors.

Mechanics checked against Physarum Pathfinding Simulation.blend and controller tests.